

METHODOLOGY

Converge

The Spine

Compile Intent into Autonomous Systems

Building Agentic Systems as an AI-Native Engineer

Fuzzy intent descends through gated passes — each bound to the right engine, each verified before the next. Today you stand at every gate. The dark factory replaces each gate with an eval. **Same passes. No human.**

Five universal passes · one fork · one gate

The thesis

An AI-Native engineer does not write the system. They **compile intent into it** — lowering a fuzzy requirement through a fixed sequence of passes until a machine can execute it without supervision. Each pass drops one level of altitude, binds the right engine for that altitude, and ends at a gate that must pass before the next begins.

The discipline is the gate. Today the gate is **you** — you confirm, you approve, you review. The dark factory replaces each human gate with a runnable eval. The passes do not change; only who stands at the gate. You remove yourself one gate at a time.

Converged means an eval passed — not that you feel done.

Structure: five passes, one fork

Passes 1–5 are the universal spine — they run on every project, identically. After the plans are hardened, the work forks: **Plan-Driven** when the trust boundary wraps the whole system, **Task-Driven** when it wraps each unit. The harness (Pass 5) is built the same either way; only the build path differs. Both forks reconverge on one gate — *eval passes = merged*.

1 Intent → 2 Structure → 3 Decomposition → 4 Consensus → 5 Harness ■ FORK
→ eval passes = merged

The passes — operational

Passes 1–4 are the thinking spine and are live as copy-paste prompt files (prompts/p1–p4). Each lowers altitude, binds an engine, and ends at a gate.

PASS 1 Intent

Understand Requirements

ENGINE	Claude Chat — conversational, no repo, no code.
PRODUCES	engineering-brief.pdf — a consensus document circulated with the team; the problem in the customer's terms. (Born as PDF for human consensus; becomes markdown when it feeds the build.)
GATE	the read-back of the problem is confirmed correct — and no technology choices
DARK-FACTORY SWAP	human confirmation → the brief's acceptance criteria

Prompts · understand → interrogate → crystallize (prompts/p1-intent.md)

PASS 2 Structure

Technical Specification

ENGINE	Claude Code — open the repo; check the spec against what exists in src/.
PRODUCES	no artifact — shared understanding held in the session. The tech-spec is written with the team from the brief; Pass 2 comprehends it against the real repo.
GATE	you can explain the whole system and it is consistent with the brief and the repo
DARK-FACTORY SWAP	human review → the component map as a checkable contract

Prompts · comprehend → ground & interrogate → confirm (prompts/p2-structure.md)

PASS 3 Decomposition

Features & Plans

ENGINE	Claude Code (Auto Mode) — same session as Pass 2.
PRODUCES	sketch/analytical-backbone.md and sketch/sentinel-engine.md — the two plans, split along the spec's own seam (Layers 1–5 vs. Layer 6).
GATE	split justified; each plan has features, deps, build order; the interface is named
DARK-FACTORY SWAP	human approval → coverage check + schema-valid plans

Prompts · decompose → plan backbone → plan sentinel (prompts/p3-decomposition.md)

PASS 4 **Consensus**

Brainstorm & Sharpen

ENGINE	Codex (the adversary) + Claude (revises) — a different model refutes the plans.
PRODUCES	the same two plans, sharpened in place, plus an open-questions list (each objection: FIXED, or ACCEPTED with an owner).
GATE	no open objection remains — every attack fixed in a plan or owned as a risk
DARK-FACTORY SWAP	human consensus → adversarial review passes

Prompts · attack → grill vs docs → sharpen (prompts/p4-consensus.md)

Pass 2 keeps no file on purpose — its output is the loaded understanding in the session, so Passes 2–4 run as one continuous Claude Code session. Pass 5 (Harness) is the one artifact that outlives the project; it and the Fork are the next phases to operationalize.

The Fork — the build

Decided at the end of Consensus (coupling is clear once the plans are sharp). Executed after Harness (which is fork-independent). The fork is the build: it contains both cutting the work and executing it.

The deciding question · Where does the trust boundary sit — around the whole system, or around each unit? Coupling decides.

FORK A		FORK B	
TOOL	SpecKit / OpenSpec / AgentSpec	TOOL	task-spec + engines
TAKE IT WHEN	one tightly-coupled system — pieces only make sense together	TAKE IT WHEN	many loosely-coupled units — each stands alone
CUT + BUILD	feed the plan to the spec framework → it generates the system as one motion	CUT + BUILD	task-spec cuts atomic tasks (each with an eval) → engines build them: Kimi (cheap S/M), Claude (judgment), Codex (adversarial)
TRUST BOUNDARY	wraps the whole system	TRUST BOUNDARY	wraps each unit
VERIFY	one end-to-end test	VERIFY	per-unit runnable evals
THIS PROJECT	Analytical Backbone — Postgres → Dagster → dbt → DuckDB → MCP is one pipeline	THIS PROJECT	Sentinel Engine — 5 agents + tools + loop are independent units

Both forks converge on the same gate: eval passes = merged. Fork A's eval is end-to-end; Fork B's are per-unit — the cut differs, the gate is identical. That shared gate is what makes Converge one method with two paths, and what lets either fork go dark-factory.

Pass 5 — Harness, and the build

After Consensus the spine continues into the build. These are the next passes to operationalize as prompt files.

Pass	What it produces	Engine
5 • Harness — Control Layer	agents, KBs, commands, rules, CLAUDE.md — the reusable build-system. The one artifact that outlives the project.	skills (agents-kbs-tech-stack)
6 • Tasking — cut the work	Fork A: one coherent spec. Fork B: many atomic task-specs, each with a runnable eval.	task-spec / SpecKit
7 • Execution — build-out	merged code; both forks rejoin at one gate (eval passes = merged).	Claude · Kimi · Codex

Grounded in current practice

Converge is a synthesis: Spec Kit / BMAD's phased descent (Passes 1–3), Harness Engineering's Control Layer (Pass 5), and Loop Engineering's autonomous endpoint — unified by one fork rule and one gate. Its original contribution is the Fork: making plan-driven vs. task-driven an explicit, coupling-based decision that no other framework treats as first-class.

The endpoint adds the loop: gated passes are the human-driven form; the dark factory wraps the passes in a self-feeding loop and replaces each gate with an eval. Automate the gates AND loop the passes, and the human is gone.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate.

You are converged when the eval passes — not when you feel done.

The Harness is the one pass that outlives the project — build once, reuse everywhere.

Remove yourself one gate at a time. What's left is the dark factory.